A Low-Privilege PowerShell Auditing Tool for Active Directory, with a Plain-Language Remediation Dashboard

Mid-Course Progress Report

Student: Chris Hoang Nghiem

Student ID: 300416011

Course: CSIS 4495, Applied Research Project, Section 2

Team Lead: Chris Hoang Nghiem (individual project, no other members)

Douglas College, Fall 2026

Demonstration video

https://www.youtube.com/watch?v=nYF1NHUkA2A

Source repository

https://github.com/Ha0n9/F26_4495_S2_ChrisN

1. Introduction

1.1 Background and context

Microsoft Active Directory is still by far the dominant identity and access management platform in enterprise IT and as such is one of the most targeted elements of the enterprise IT infrastructure. A joint cyber security advisory from the Australian Signals Directorate's Australian Cyber Security Centre, the US Cybersecurity and Infrastructure Security Agency, the US National Security Agency, the Canadian Centre for Cyber Security and the UK and NZ national cyber security centres describes the Active Directory attack surface as extremely large and difficult to defend and lists seventeen techniques that are commonly used to attack Active Directory environments (asd ACS et al, 2024) including Kerberoasting, unconstrained delegation abuse, DCSync and golden ticket attacks.

For small and medium enterprises, Active Directory is often run without having an active security department. In such cases, while the IT staff of these organizations may be highly skilled in handling everyday AD management, the same may not be true when it comes to understanding the results generated by security audit tools.

1.2 Related work and the gap this project addresses

There are several known techniques for evaluating the security posture of Active Directory. For example, PingCastle, developed by Vincent Le Toux and supported by Netwrix, is one such tool that does a risk-scored health analysis in areas like stale objects, privilege management, trusts, and anomalies (Le Toux & Netwrix, n.d.). BloodHound is another tool that uses a different technique to evaluate security posture by showing the attack paths by which an attacker could leverage privileges from existing configurations to become a Domain Administrator (Granneman, n.d.).

The problem with both of these is that they are effective, and that they are more thorough than any tool produced by this project. The difference is in the audience. Attack-path graphs and configuration risk matrices presume that the person reading them understands what Kerberoastable service accounts and unconstrained delegation flags mean, and what needs to be done about it. An administrator who lacks this knowledge will be able to use either tool and get a correct output but will have no clue how to fix anything.

The next difference is about privilege. Audit tools need high privilege to be launched due to the simplicity of such approach. An audit tool which needs full control over the domain in order to audit the security of that domain is an element of danger by itself because the account it needs is just what a hacker wants to get.

1.3 Research questions

This project addresses two questions:

- Can an open, PowerShell-native auditing tool detect a representative set of common Active Directory misconfigurations and present each finding such that a generalist administrator,

without dedicated security training, can understand and act on it without first consulting a specialist?

- Can such a tool operate usefully from an ordinary, low-privilege domain account, reporting honestly on what it was unable to examine rather than silently treating inaccessible ground as clean?

1.4 Hypotheses and assumptions

- A representative subset of well-documented misconfigurations can be detected using native PowerShell and the ActiveDirectory module alone, without a compiled binary or third-party agent.
- Separating detection from explanation, so that the module emits only structured data and a separate content layer supplies the human-readable text, will make both halves independently maintainable, and will allow the detection engine to be used headless from the command line.
- Framing each finding as what is wrong, why it matters, and what to do will make the output more actionable for a non-specialist than a technical report.
- Because audit output reveals sensitive infrastructure information, including account names, group structure and delegation paths, the presentation layer must run entirely locally with no cloud dependency.

1.5 Expected contribution

For a small IT team without a dedicated security function, the expected contribution is a free, self-hostable first line of Active Directory assessment that lowers the expertise barrier to acting on findings, and that can be run by an account which is not itself a high-value target. For me, the project is a deliberate bridge from a web development background toward SOC analyst and security engineering work, deepening PowerShell, Windows-based defensive security and Active Directory internals.

2. Summary of the Initially Proposed Project

The approved proposal included the implementation of a PowerShell module that would scan an Active Directory domain to identify security misconfigurations from eight different categories. Also, a web dashboard on the local machine was proposed to translate the raw technical data into plain English, along with severity levels and remediation measures. The eight checks included in the proposal were: expired/inactive users, weak password policies, privileged group membership, Kerberoasting, unconstrained delegation, risky group policies, domain trusts, and LAPS installation status.

The architectural approach taken with respect to the proposal is one that splits detection from presentation. The PowerShell module runs the audit process and generates data in JSON format, which means that the auditing tool can stand alone without being reliant on the command line. There is also a conversion layer that turns technical information into user-friendly information.

Additionally, this proposal promised the inclusion of three criteria of evaluation which included accuracy of the detection process by using ground truth created on purpose in a laboratory environment, portability through testing the developed module in a second, but different laboratory environment and finally, the usability of the information provided for remediation as understood by someone who does not have any experience in Active Directory security management.

The proposed schedule placed a minimum viable product at week four: a complete scan to JSON to dashboard workflow with at least three security checks implemented end to end.

3. Changes to the Proposal

Five substantive changes have been made. Each is stated with the reasoning that led to it.

3.1 Storage: SQLite replaced by timestamped JSON files

This plan mentioned that the potential way of storing historical scan data would be the local SQLite database; JSON snapshots could serve as an alternative option. The latter solution was not implemented because of dropping the database.

Rationale: A persistence mechanism would have necessitated schema migrations, a query mechanism and increased maintenance costs in return for trend tracking functionality, which is not needed by any of the criteria in the evaluation of the proposal. Local timestamped JSON files meet all present needs and keep the module isolated. SQLite still can be used as an option in the last phase if trend tracking emerges as a need.

3.2 Execution model: a dedicated low-privilege account on a separate workstation

There was no mention about the required privilege level at which the tool should operate, and the underlying assumption at the planning stage was that the scan would take place on the domain controller. In reality, the implemented solution differs significantly as the scan takes place from the Windows 11 workstation connected to the domain but signed in as a regular domain user.

Justification: This is the modification that I finds most justifiable in terms of security. The use of an auditing software that needs Domain Administrator rights does not minimize risks; it actually aggregates them. In this proposed model, a Domain Administrator gives the audit account the right to view any of those that they want to be audited, and the software will report with transparency any area it was unable to access. Another benefit is that the audit machine is isolated from the domain controller.

The redesign helped make the reporting design stronger. This is because there will always be directories that a low-privilege account will not be able to see. Therefore, the difference between a query that found nothing and one that couldn’t run at all became a first-class notion in the JSON output and on the dashboard.

3.3 A local API replaces the planned launcher script, pulled forward from weeks nine and ten

The proposal had an action plan for the launching script during the packaging phase which would perform the scanning process, launch a local server and then launch the web browser. This has now been superseded by a local HTTP API developed in the fourth week rather than in the ninth week.

Justification: The mid-term presentation course requirement entails the end-to-end process starting from the interface and not a series of scripts that will run on the command line. A launcher script would meet the ease aspect but fail to meet the presentation aspect of the course requirement. Additionally, the API eliminates the need to copy the scan results manually to the directory where the dashboard is serving its contents.

This added a security concern which did not previously exist. The endpoint which has the capability of executing the PowerShell script on demand is accessible by any page loaded on the user's browser. This requires the following conditions all together: the listener should bind only to the loopback interface; the request should include a token generated during server startup and displayed on the console; and the Origin header, if set, should be equal to any one from the dashboard. Parameters for the scan are sent as individual parameters to the PowerShell script file instead of including them in a command line.

3.4 A translation layer written in a third language was considered and rejected

During week four I have considered implementing the plain-language translation layer as a separate Python component that would parse the module's JSON and compile it into readable output.

Justification for rejecting it: the proposal's central claim is that the tool must be installable and reusable by another administrator. The implementation already requires PowerShell and Node.js. Adding Python would have introduced a third runtime and a third installation barrier. More importantly, the translation layer already existed in the dashboard, as a static content dictionary joined to findings on check identifier, so a Python implementation would have duplicated working logic and created two places to maintain the same wording. The risk I was attempting to mitigate, that translation might lose context or produce vague output, is a content quality risk rather than an engineering one, and is addressed by the usability evaluation already committed to in the proposal.

3.5 Check sequencing revised, with an explicit cut order

The proposal scheduled all five remaining checks across weeks five and six. Three checks required approximately four weeks of elapsed effort; five checks in two weeks is not a credible estimate against the same constraints.

Justification and revised sequence: the proposal anticipated this, stating that if the full set proved too broad, checks would be prioritized by security impact, favouring items identified in the joint advisory. Kerberoasting exposure and unconstrained delegation are therefore scheduled first, as both are named in that advisory. Risky Group Policy configurations and domain trust relationships follow. The LAPS deployment check is scheduled last and is the first item to be cut if the schedule does not permit it. Any check not delivered will be stated explicitly in the final report with the reasoning above, rather than quietly omitted.

3.6 An observation on sequencing, offered without justification

Within the first three weeks, the ordering of work diverged from the plan in a way worth recording honestly. The React dashboard was delivered approximately two weeks ahead of its scheduled week, while the PowerShell module and the lab environment were delivered approximately two weeks behind theirs.

The explanation is straightforward: the dashboard sits within my existing competence, and the PowerShell and Active Directory work does not. Work gravitated toward the familiar. The week four gate was met regardless, but the pattern is noted here because the remaining schedule is composed almost entirely of unfamiliar work, and the same drift would be more costly now than it was in October.

4. Project Planning and Timeline

This is an individually completed project. Chris Hoang Nghiem is the sole author and is responsible for every milestone and deliverable below. No work is shared with or delegated to another person.

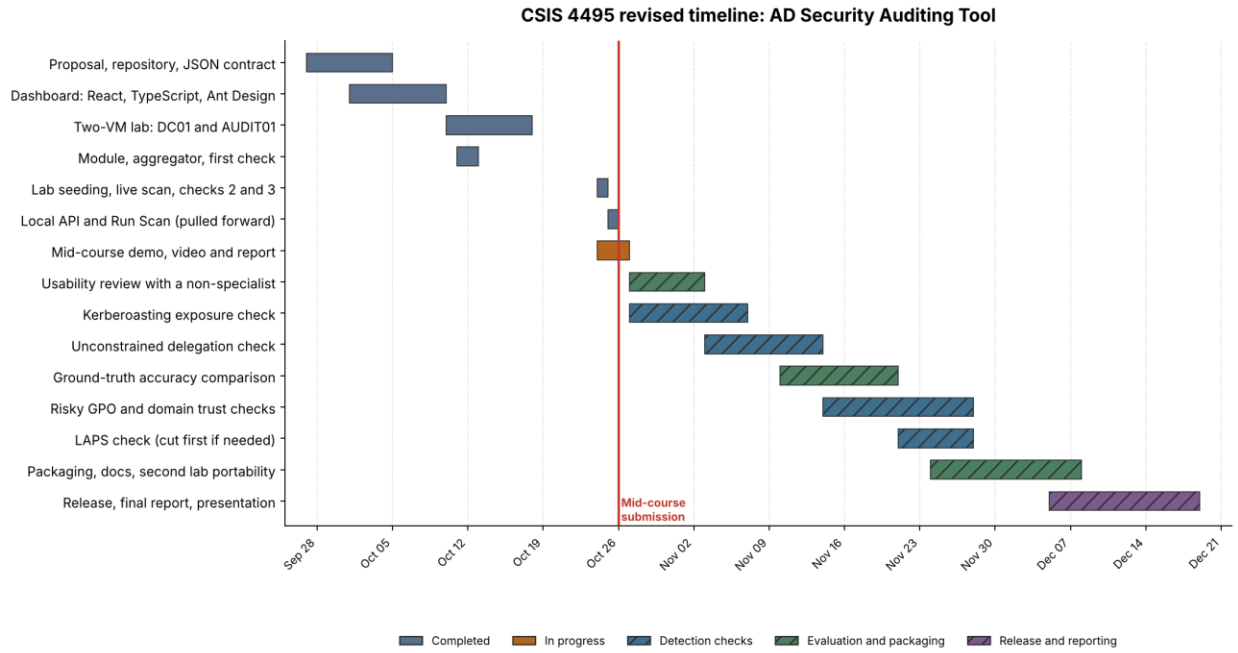
4.1 Position against the original schedule

Week one began on 27 September 2026. The mid-course submission falls at the start of week five.

Milestone	Due	Actual	Assessment
Week 1	3 Oct	JSON contract and check list complete; module scaffold delivered 12 Oct	Module scaffold nine days late
Week 2	10 Oct	Lab built 15 Oct, seeded 24 Oct. Checks delivered 12, 24 and 25 Oct	Approximately two weeks late
Week 3	17 Oct	Dashboard delivered 2 Oct	Approximately two weeks early
Week 4	24 Oct	Full chain working 24 Oct; three checks complete 25 Oct	**Met**
Weeks 9 to 10	5 Dec	Local API delivered 25 Oct, exceeding the planned launcher script in scope	Approximately six weeks early

The binding gate for this submission was the week four milestone, which required a complete scan to JSON to dashboard workflow with at least three checks implemented end to end. That milestone has been met.

4.2 Revised schedule to end of term



Weeks	Deliverable	Evidence of completion
5 (27 Oct to 2 Nov)	Kerberoasting exposure check. Usability review: one person without Active Directory security expertise reads a sample of findings and reports whether they are actionable.	Check committed with its verification script; written notes from the usability review
6 to 7 (3 to 16 Nov)	Unconstrained delegation check. Automated comparison of scan output against ground-truth.json, producing false positive and false negative counts per check.	Check committed; Compare-AuditResult function and its first measured output
8 to 9 (17 to 30 Nov)	Risky Group Policy and domain trust checks. LAPS check if the schedule permits. Second lab domain built with a different configuration.	Checks committed; second lab documented
10 to 11 (1 to 14 Dec)	Module packaging with manifest and versioning. Portability run against the second lab without source modification. Documentation reference for every check.	Installable module; portability result recorded
12 (15 to 19 Dec)	Public release, final report, presentation.	Published repository; submitted report

Two risks are embedded in this schedule. The first is that the remaining checks are the ones which are technically the most difficult and are outside my current competence (the cut order in section 3.5 is specifically to deal with that). The second is that both the usability evaluation and the portability test are going to be dependent on resources which are outside my direct control (i.e. another person's time and a second lab). The usability review is therefore scheduled early rather than late.

5. Implemented Features

All the work described in this section was designed, implemented and tested by me (who is the only member of the project). Section 7 details where generative AI was used and what I did in addition to that.

5.1 Detection engine and scan envelope

The PowerShell module is organised as a manifest, a loader that dot-sources every function and exports only the public surface, a finding constructor that defines the JSON contract in exactly one place, and an aggregator that runs the registered checks and assembles the result.

The aggregator holds three responsibilities and deliberately contains no detection logic of its own: deciding which checks to run, running each inside isolated error handling, and assembling the scan envelope. The isolation is the significant part. A check that throws is recorded under checksFailed with its error message and excluded from checksCompleted, so a scan that could not examine part of the domain reports that fact rather than presenting a partial result as a complete one.

The module computes no security score. Scoring is a presentation concern and lives in the dashboard, so the weighting can be revised without re-running a scan against a production domain.

5.2 Implemented detection checks

Three checks are complete and have been run against the live lab domain from an ordinary domain account.

Check	Identifier	What it reports
Weak domain password policy	AD.POLICY.WEAK_PASSWORD	Minimum length, complexity, lockout threshold and reversible encryption, compared against a caller-supplied baseline rather than hardcoded values
Stale enabled accounts	AD.ACCOUNT.STALE_ENABLED	Enabled accounts that have never signed in, or that have been inactive beyond a threshold, as one finding covering all of them
Excessive privileged group membership	AD.PRIVILEGE.EXCESSIVE_ADMIN_MEMBERSHIP	Accounts holding domain-wide privilege, including membership inherited through nested groups, with the chain the privilege arrived through

Design decisions within the checks

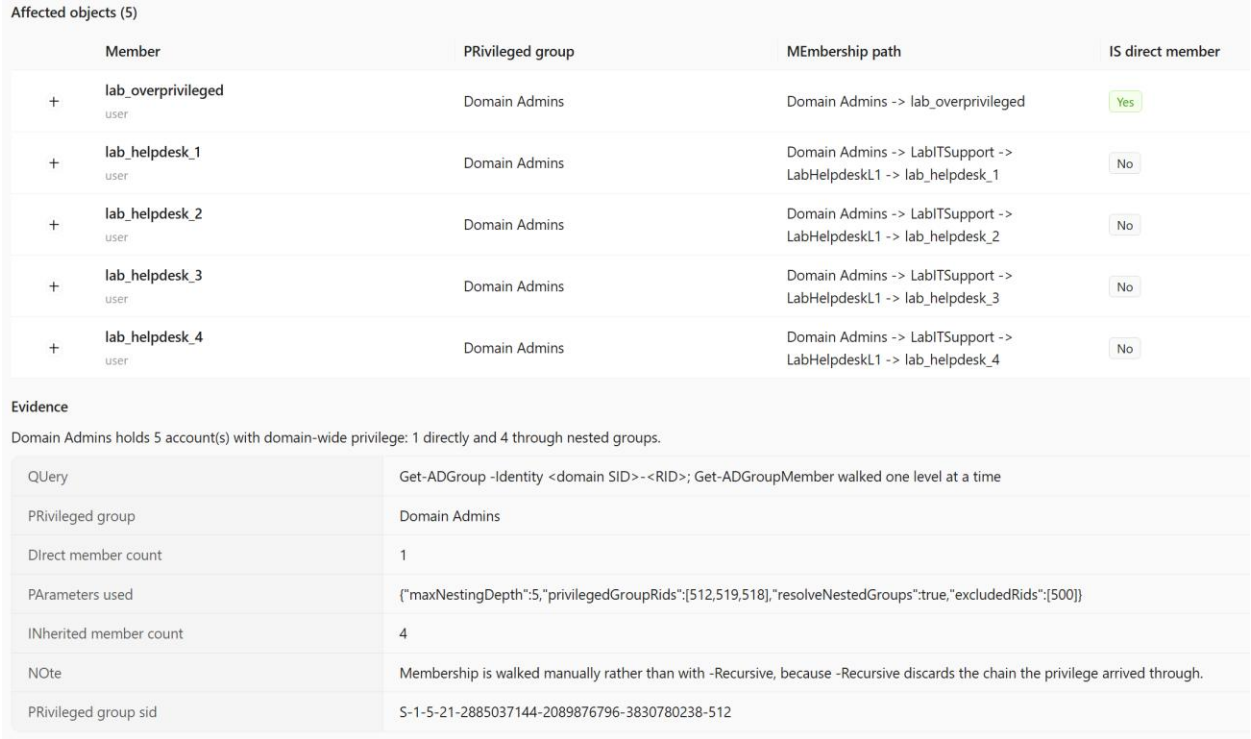
Three decisions are worth recording because each was reached by investigating behaviour that is not obvious from the cmdlet documentation.

Privileged groups are resolved by security identifier rather than by name. The group name Domain Admins is localised; a German-language domain names it Domänen-Admins. A name-based lookup on such a domain returns nothing and the check would then report a clean result, which is the most dangerous possible failure mode for a security tool. The relative identifier, 512, is invariant across

domains. Portability across domains is an explicit evaluation criterion in the proposal, and this is where it is won or lost.

Nested membership is walked one level at a time rather than with the Recursive switch. The switch flattens the hierarchy and returns leaf objects efficiently, but discards the chain through which the privilege was inherited. Without that chain an administrator learns that a user holds domain-wide privilege and has no indication of which group to modify. The check therefore performs its own breadth-first traversal, carrying the path, with a visited set guarding against membership cycles, which Active Directory permits and which cause a naive recursion never to terminate.

Accounts that have never signed in are reported with a null last-logon value rather than a very old date. An account never used and an account unused since 2023 call for different conversations with the account owner, and collapsing both into a date destroys that distinction. A related limitation is recorded in the finding evidence itself: the LastLogonDate property derives from lastLogonTimestamp, which Active Directory replicates only every nine to fourteen days by default, so inactivity figures are approximate to within that window.



Affected objects (5)

	Member	PRivileged group	MEmbership path	IS direct member
+	lab_overprivileged user	Domain Admins	Domain Admins -> lab_overprivileged	Yes
+	lab_helpdesk_1 user	Domain Admins	Domain Admins -> LabITSupport -> LabHelpdeskL1 -> lab_helpdesk_1	No
+	lab_helpdesk_2 user	Domain Admins	Domain Admins -> LabITSupport -> LabHelpdeskL1 -> lab_helpdesk_2	No
+	lab_helpdesk_3 user	Domain Admins	Domain Admins -> LabITSupport -> LabHelpdeskL1 -> lab_helpdesk_3	No
+	lab_helpdesk_4 user	Domain Admins	Domain Admins -> LabITSupport -> LabHelpdeskL1 -> lab_helpdesk_4	No

Evidence

Domain Admins holds 5 account(s) with domain-wide privilege: 1 directly and 4 through nested groups.

QUery	Get-ADGroup -Identity <domain SID>-<RID>; Get-ADGroupMember walked one level at a time
PRivileged group	Domain Admins
DIrect member count	1
PArameters used	{"maxNestingDepth":5,"privilegedGroupRids":[512,519,518],"resolveNestedGroups":true,"excludedRids":[500]}
INherited member count	4
NOte	Membership is walked manually rather than with -Recursive, because -Recursive discards the chain the privilege arrived through.
PRivileged group sid	S-1-5-21-2885037144-2089876796-3830780238-512

screenshot of the expanded privileged membership finding, with the membership path column visible.

5.3 The translation layer

The plain-language content lives in a hand-written dictionary file, keyed by check identifier, entirely separate from the detection code. Each entry supplies a title in ordinary language, an explanation of why the issue matters, an ordered list of remediation steps, a default severity, and the attribute names that should be promoted into table columns.

The module never writes explanatory or remediation text. The rule applied throughout is that if a string would read like advice to an administrator, it belongs in the dictionary. This keeps the module independently usable from the command line and means the wording can be revised without touching detection logic or re-running a scan.

The contract between the two halves carries a rule that protects the user from an incomplete content layer: a finding whose check identifier is absent from the dictionary is still rendered, using the identifier as its title. Findings are never hidden because the explanation is missing. During implementation this rule was found to be violated at a larger scale, since a failure to load the dictionary file prevented every finding from rendering, and the loading logic was corrected so that a missing dictionary degrades to identifiers plus a warning rather than to a blank page.

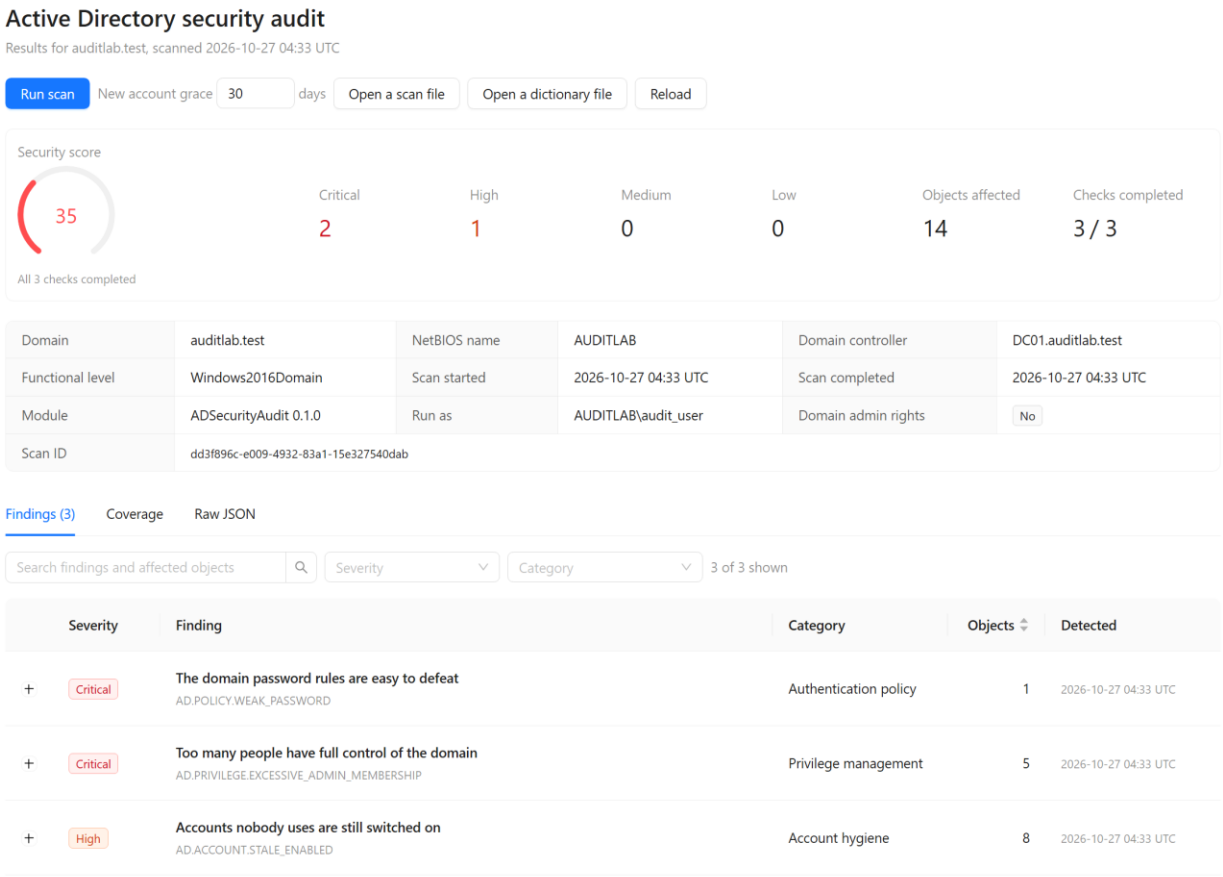
5.4 The dashboard

The dashboard is a Vite, React and TypeScript application using Ant Design components, running locally only. It presents three views.

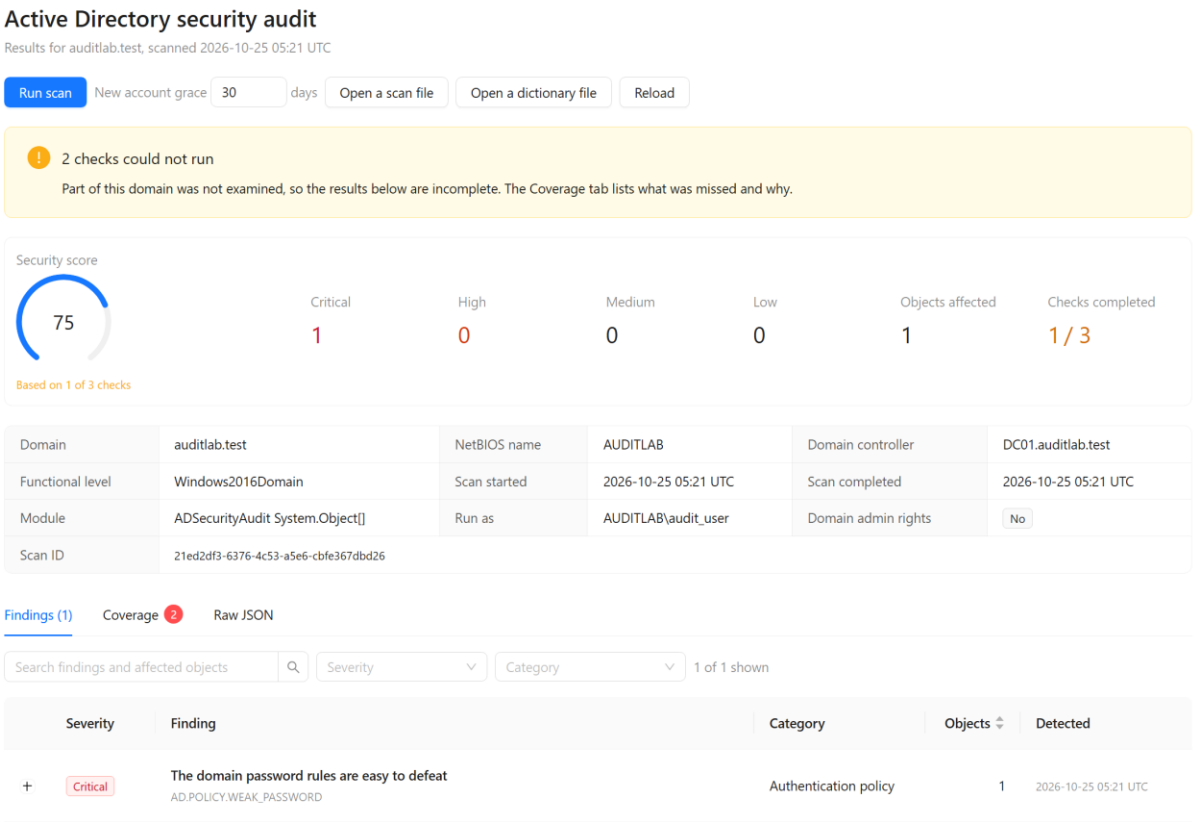
- Findings, grouped by severity and filterable, each expanding to show the plain-language explanation, numbered remediation steps, a table of affected objects, and the raw evidence the check gathered.
- Coverage, which lists every requested check and separates those that completed from those that could not run, with the reason.
- Raw JSON, for inspecting exactly what the module produced.

The security score is computed from the whole scan rather than from the findings alone. An earlier implementation derived it from findings only, which meant a scan in which every check failed produced zero findings and therefore a perfect score of one hundred, displayed alongside a tab stating that nothing had been examined. The corrected implementation returns no score at all when no check

completed, and otherwise states how many checks the score is based on.



screenshot of the dashboard showing three findings, score 35, and 3 of 3 checks completed.



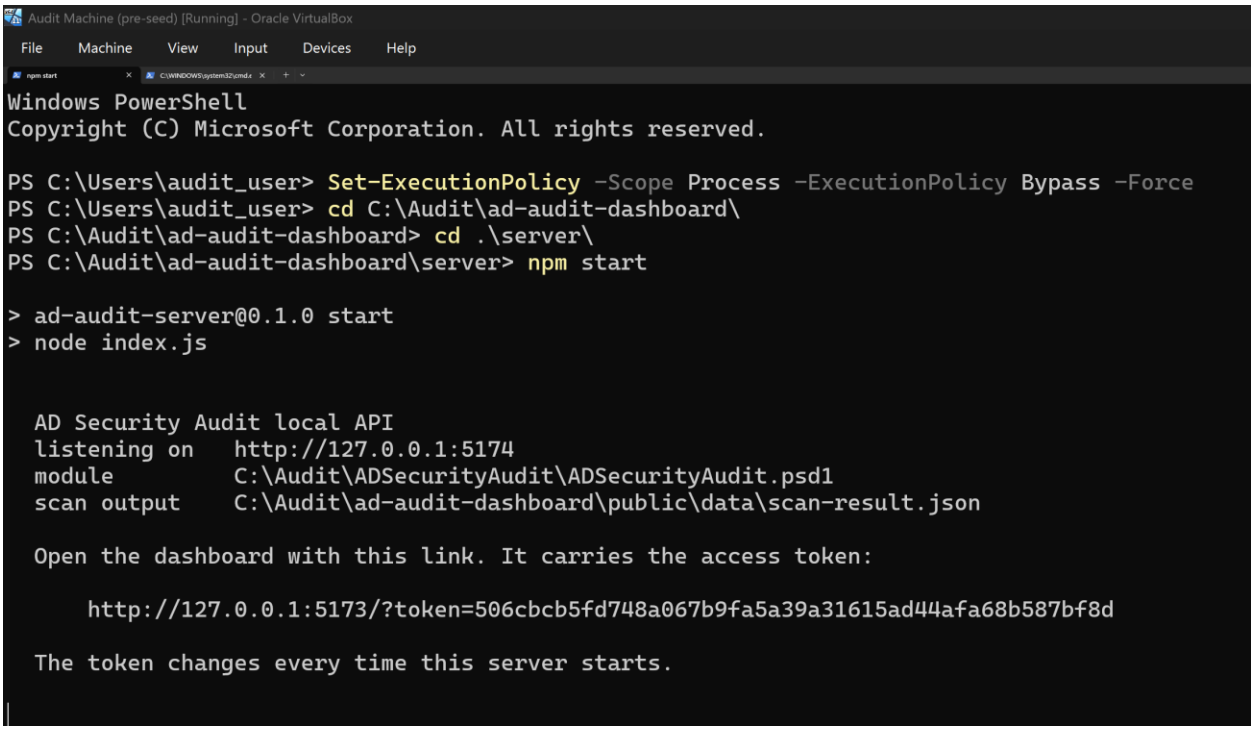
screenshot of the Coverage tab from the earlier scan in which one check could not run

5.5 The local API

The API is a small Express server that allows the dashboard to start a scan. It exposes four endpoints: a health check, the finding dictionary, the most recent scan result, and a scan request. A scan request spawns PowerShell, runs the module against the domain, and returns the resulting object together with the elapsed time and any warnings the module emitted.

Because this process runs PowerShell on request, it is protected by three measures that must all hold: the listener binds the loopback interface only; every request must present a token generated at startup and printed to the console; and the Origin header, where present, must match one of the dashboard's own. The token is delivered to the browser in the dashboard URL and removed from the address bar once read, so it does not persist in browser history.

Scan arguments are not interpolated into a command string. They are passed as discrete arguments to a PowerShell file on disk, which removes an entire class of quoting and injection problems that would otherwise arise from a value crossing Node argument handling, Windows command parsing and PowerShell parsing in sequence.



screenshot of the API console output at startup, showing the tokenised link

5.6 The test lab and ground truth

The lab consists of two virtual machines on an isolated network: a Windows Server 2022 domain controller hosting auditlab.test, and a Windows 11 workstation joined to that domain with only the remote server administration tools installed. The audit account is an ordinary domain user in a dedicated organizational unit, with no administrative group memberships.

Misconfigurations are introduced by a seeding script rather than by hand. The script supports a dry run, generates a separate random password for every account it creates, and writes a ground truth file recording exactly what it seeded. Reproducibility matters here for two reasons: the lab can be rebuilt after a failed experiment, and detection accuracy can be measured against a record produced by the same process that created the conditions.

5.7 Automated testing

Sixty assertions cover the two most recent checks, executed against stand-in implementations of the Active Directory cmdlets so that detection logic can be verified on any machine without a domain. The stand-ins are defined whether or not the remote administration tools are present, which serves two purposes: without them the tests could not run at all, and with them a unit test able to reach a live domain is a unit test that might accidentally query one.

The assertions encode the behaviours identified in section 5.2 as easy to get wrong. One asserts that the privileged membership check never invokes Get-ADGroupMember with the Recursive switch. Another

constructs a membership cycle and requires both that the traversal terminates and that a user beyond the cycle is still found.

Writing these tests surfaced a latent defect in the shared timestamp helper, which had been unreachable for two weeks because no completed check had passed a non-null timestamp through it. Two further defects were found only when real scan output was rendered for the first time: a timestamp formatter that converted to local time while still labelling the result as UTC, and an attribute heading function that capitalised the first character twice, producing headings such as MIn password length.

```
PS C:\Audit\ADSecurityAudit\Tests> .\verify-admin-check.ps1
```

```
Membership cycle
  pass  terminates instead of recursing forever
  pass  still finds the user beyond the cycle

Several privileged groups
  pass  one finding per group
  pass  distinct finding ids

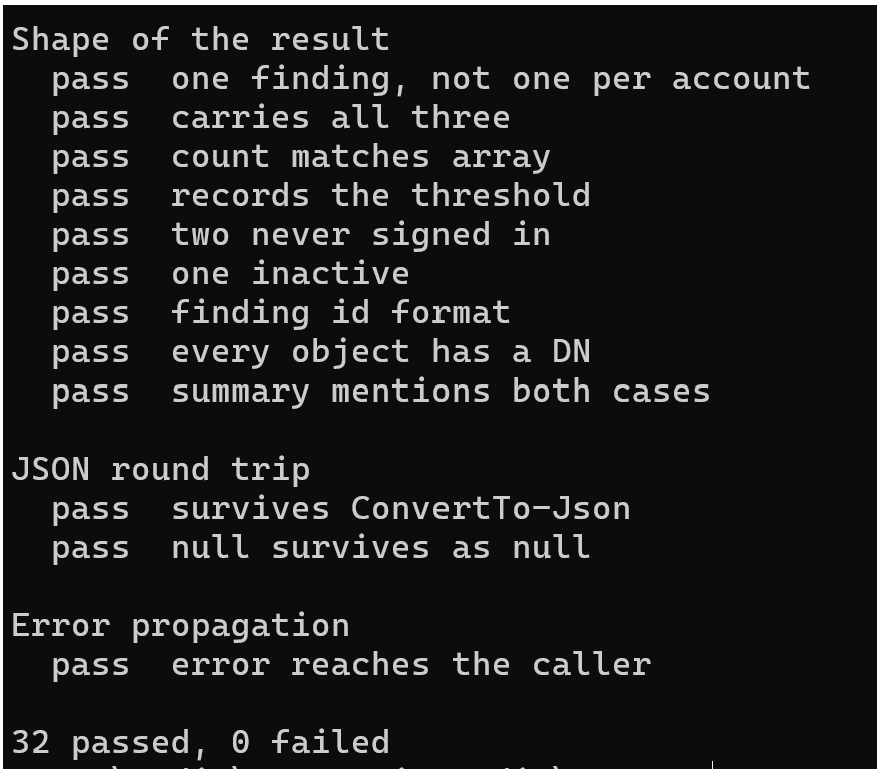
Group absent from this domain
  pass  a missing group is skipped, not an error

JSON round trip
  pass  survives ConvertTo-Json

Error propagation
  pass  a hard failure reaches the caller

28 passed, 0 failed
```

```
PS C:\Audit\ADSecurityAudit\Tests> .\verify-stale-check.ps1
```



screenshots of both verification scripts running, showing 32 and 28 assertions passing

6. AI Use

6.1 Tools and specific use

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition
Claude (Anthropic)	Claude Pro – Opus 5	Generated the PowerShell module scaffold, the finding constructor, the aggregator, the lab seeding script, the local API server, the React dashboard scaffold and components, the test harnesses, and drafts of the project documentation. Also used to review the repository against the course requirements and to identify defects in generated code.	Claude helped me produce initial versions of the project's code, tests, and documentation, which reduced the time needed to build a working prototype. My contribution was to review the generated work, run it against my own Active Directory lab, report the results, and make decisions about the project after discussing the available options. I built and configured the two-VM lab myself, including the isolated network, domain controller, audit workstation, low-privilege account, and RSAT tools. I also investigated the domain join and Active Directory Web Services issues that arose during setup. Testing the

			generated PowerShell against the live lab helped uncover defects in file output, module version reporting, and timestamp conversion. By examining the actual dashboard output, I also identified display and data-loading problems. After discussion, I decided to use a low-privilege account on a separate audit workstation and not to add a Python translation layer. I also decided to complete the third check despite a suggestion to defer it. These decisions, together with my testing and verification, helped turn the generated components into a working project that I could evaluate in my own environment.
GitGuardian	Free base plan	Automated secret scanning on the GitHub repository. Detected a hardcoded password committed in the lab seeding script.	GitGuardian alerted me to a hardcoded password that had been committed in the generated lab seeding script. The alert helped me recognize that changing the script in a later commit alone would not remove the exposed password from the repository history. I therefore had the script changed to generate random passwords and removed the secret from the Git history. This gave me practical experience responding to a security issue during development.

6.2 Appendix: prompt history

The following appendix summarizes the prompts used during the development of this project. The conversations were primarily conducted in Vietnamese, and the English descriptions below reflect the topics and requests recorded in the available prompt history. Some screenshots, terminal output, error messages, and longer passages are represented by summaries.

Phase 1: Project selection and proposal

The initial prompts focused on selecting a project that could be completed within twelve weeks and developed into a usable, potentially open-source tool. I discussed my limited experience with PowerShell, the instructor's requirement for a working demonstration, and the need for a user interface that would present findings in a more accessible form than raw JSON.

Examples of requests included:

- “What kind of project would be reasonable to complete within twelve weeks?”
- “Write a preliminary project plan in English.”
- “Rewrite the idea as an architecture description and a basic implementation plan.”
- “Summarize the steps for building an Active Directory environment with test misconfigurations and a basic dashboard.”

Phase 2: Dashboard design and implementation

I specified that the frontend should use an existing component library, such as Ant Design or shadcn/ui, rather than custom-designed components. I also set visual constraints prohibiting gradients, glassmorphism, nested cards, and excessive shadows. After reviewing an initial demonstration, I asked for the frontend to be developed as a TypeScript project.

Examples of requests included:

- “Use an existing component library for the frontend and avoid creating custom component designs.”
- “Do not use gradients, glassmorphism, nested cards, or shadows throughout the interface.”
- “Build the frontend first.”
- “Turn the initial demonstration into a TypeScript project.”

Phase 3: Repository structure, tests, and Progress Report 1

The prompts in this phase concerned organizing the repository according to the course requirements, drafting the first progress report, and preparing the PowerShell module and tests. I also supplied Git output and a GitGuardian alert for troubleshooting.

Examples of requests included:

- “Review the progress report template and reorganize the repository according to the instructor’s requirements.”
- “Review the repository commits and help me draft the progress report.”
- “Write tests for the stale enabled account and excessive privileged group membership checks so that I can study their implementation.”
- “Help me address the hardcoded password reported by GitGuardian and remove it from the Git history.”
- “Set up the Active Directory environment so I can test the project directly.”

Phase 4: Architecture and proof of concept

I discussed what the proof of concept needed to demonstrate and reviewed the responsibilities of the existing modules. I considered a Python-based translation layer and asked how the PowerShell output would reach the dashboard. The discussion also addressed the instructor’s requirement for a visible end-to-end workflow.

Examples of requests included:

- “Review the repository and explain the function of each module.”

- “What should the proof of concept demonstrate for the project?”
- “I am considering a Python component to translate the audit output into readable information.”
- “The instructor wants an end-to-end demonstration connecting the backend to the frontend. How should the components work together?”

Phase 5: Live testing, checks, and local API

I ran the generated scripts in the lab and provided terminal output, error messages, and screenshots for troubleshooting. This included problems with file output, execution policy, verification scripts, and dashboard display. I also requested implementation of additional checks and a local API to connect the interface to the PowerShell module.

Examples of requests included:

- “Provide detailed, ordered instructions for running the project in the lab.”
- “Help diagnose the Set-Content error shown in the PowerShell output.”
- “Implement the stale enabled accounts check and explain how to run its verification script.”
- “Implement the third check as well.”
- “Why do some dashboard headings have two capital letters at the beginning?”

Phase 6: Documentation and final review

The final prompts focused on comparing the implementation with the proposal, updating documentation, preparing the demonstration, and troubleshooting the project before submission. I also provided screenshots and output from the running application to identify issues.

Examples of requests included:

- “Compare the repository with the proposal and assess the project’s progress.”
- “Update the Markdown documentation to match the current implementation.”
- “Write a narration script for the demonstration video and explain the token displayed by the backend.”
- “Help troubleshoot the API connection and the blank dashboard when loading a scan file.”
- “Check whether the proposed screenshots support the relevant sections of the mid-course report.”

Note: This is a summary of the available prompt history. Some entries refer to screenshots, pasted terminal output, or attached files whose full contents are not reproduced above.

7. Work Date and Hours Log

Student name: Chris Hoang Nghiem

Date	Hours	Description of work done

Sep 27	1	Drafted the project proposal; created the GitHub repository; committed the proposal
Sep 27	1	Revised the proposal after review
Sep 28 to Oct 1	5	• Designed the JSON contract between the PowerShell module and the dashboard, defined the finding schema (checkId, severity, affectedObjects, evidence) and wrote CONTRACT.md • Built the finding dictionary as a separate content layer, so detection logic and plain language remediation stay independent, wrote entries for the first three checks • Evaluated Ant Design against shadcn/ui for the dashboard and chose Ant Design for its ready-made data table, severity tags and expandable rows; scaffolded the Vite, React and TypeScript project • Built the dashboard data layer: TypeScript types mirroring the JSON contract, and the join between findings and dictionary entries on checkId
Oct 2	4	Built dashboard components for findings display; committed to repository
Oct 3 to Oct 9	19	• Researched Active Directory fundamentals and identified the information needed for the project's audit checks. • Investigated the behaviour of Active Directory PowerShell cmdlets used to retrieve users, groups, and their properties. • Researched the lastLogonTimestamp attribute, including how it is updated and replicated between domain controllers. • Compared lastLogonTimestamp with lastLogon to understand their limitations when identifying inactive accounts. • Investigated nested group membership and how to resolve direct and indirect members of Active Directory groups. • Researched how privileged group names can differ across Windows language settings and considered ways to identify groups reliably. • Consolidated the research findings and documented considerations for implementing the audit checks. All work during this period was performed locally and was not committed to the repository.
Oct 10	2.5	Restructured the repository to the course layout; added the repository README
Oct 10	4	Wrote the lab seeding script with ground truth output
Oct 12	1	GitGuardian flagged a hardcoded password in the seeding script; rewrote it to generate per-account random passwords using a cryptographic RNG; purged the literal from git history with git filter-repo and verified with git log -S
Oct 12	2	Committed the module structure: manifest, loader, finding constructor, aggregator and JSON envelope
Oct 12	0.5	Implemented the weak domain password policy check
Oct 13 to Oct 15	10.5	• Created the two VirtualBox virtual machines for the project: DC01 running Windows Server 2022 and AUDIT01 running Windows 11. Configured the isolated host-only network and assigned static IP addresses to both machines. • Installed Active Directory Domain Services on DC01 and promoted it to a domain controller for the auditlab.test domain. Configured DNS and tested connectivity between the two machines. Diagnosed and resolved the domain join issue, then joined AUDIT01 to the domain. • Created a low-privilege domain account for running the audit. Installed the Active Directory RSAT tools on AUDIT01 and investigated the Active Directory Web Services error encountered in an elevated execution context. Confirmed

		that the AD commands worked when using the auditor account, then created snapshots of both virtual machines.
Oct 24	2	Applied dashboard fixes; ran the seeding script against the lab; executed the first scan against the live domain from AUDIT01 as an ordinary user
Oct 24	1	Diagnosed a defect where the module reported success for a file it had not written, because Set-Content fails non-terminating when the output directory is absent
Oct 24	3	Implemented the stale enabled accounts check with its verification script, 32 assertions
Oct 25	3	Built the local API server: four endpoints, token authentication, origin checking, and discrete-argument invocation of PowerShell
Oct 25	2	Wired the dashboard to the API: API client, Run scan control, adjustable grace period, dev server proxy
Oct 25	0.5	Implemented the excessive privileged group membership check with its verification script, 28 assertions
Oct 25	0.2	Fixed attribute heading capitalisation and corrected the independent loading of scan result and dictionary
Oct 25	1.5	Brought all documentation back into line with the implementation
Oct 26	5	Recording the demonstration video and completing this report

8. Closing and References

8.1 Acknowledgements

I want to say thanks the CSIS 4495 course instructor for guidance on scoping this project toward a reusable product rather than a one-time laboratory demonstration, and for the requirement that the mid-course demonstration show an end-to-end flow driven from the interface, which prompted the API work described in section 3.3.

8.2 References

References follow APA 7th edition style, with hanging indents and alphabetical ordering by author.

Australian Signals Directorate's Australian Cyber Security Centre, Cybersecurity and Infrastructure Security Agency, National Security Agency, Canadian Centre for Cyber Security, National Cyber Security Centre (UK), & National Cyber Security Centre (NZ). (2024). *Detecting and mitigating Active Directory compromises*. Cybersecurity and Infrastructure Security Agency. https://www.cisa.gov/resources-tools/resources/detecting-and-mitigating-active-directory-compromises

Granneman, J. (n.d.). *How the BloodHound tool can improve Active Directory security*. TechTarget. Retrieved October 25, 2026, from https://www.techtarget.com/cybersecurity/tip/How-the-BloodHound-tool-can-improve-Active-Directory-security

Le Toux, V., & Netwrix. (n.d.). *PingCastle* [Computer software]. GitHub. Retrieved October 25, 2026, from https://github.com/netwrix/pingcastle

Microsoft. (n.d.). *Active Directory Domain Services documentation*. Microsoft Learn. Retrieved October 25, 2026, from https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/active-directory-domain-services

Netwrix. (n.d.). *PingCastle vs. BloodHound: AD attack path analysis*. Netwrix Blog. Retrieved October 25, 2026, from https://www.netwrix.com/en/resources/blog/pingcastle-vs-bloodhound/